ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 10: Asynchronous Processing with Event-Driven Architecture

CO/PO	CO4 / PO3, PO5
Objective	To implement asynchronous background processing using Node.js native EventEmitter without external dependencies.
Prerequisites	• Practicals 4–9 completed working Node/Express/MongoDB backend with caching • Basic understanding of synchronous vs asynchronous execution in JavaScript

Theory Concepts (Self-Study Reference)

Coursera Pointer	No dedicated Coursera module for this topic refer to the Node.js official documentation on Events (nodejs.org/en/docs/guides/event-loop-timers-and-nexttick), supplemented by the Node.js Crash Course on the Traversy Media YouTube channel.

Architecture / Diagram

Diagram

```
POST /tasks request
        |
        v
  Save task to MongoDB
        |
        v
  emit('task-created', taskData)  ──► API responds immediately (201)
        |
        v  (handled asynchronously, after response sent)
  Notification Listener
        └── logs: task title, timestamp, assigned user
```

Problem Definition

Problem Statement	• Add a background notification system to the Task Management application using Node.js built-in EventEmitter. • When a new task is created via the API, emit a task-created event handled asynchronously. • The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response. • The main API response must return immediately, before the event handler finishes executing. • Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.
Key Questions / Analysis	• Why does emitting an event not block the API response, even though both run on the same Node.js process? • What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler? • Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?
Supplementary Problems	• Add a second event, task-deleted, with its own listener that logs a different message. • Add an error event listener that catches and logs any error thrown inside the task-created handler.

	• Simulate a slow notification handler (e.g., setTimeout of 2 seconds) and confirm the API still responds instantly.
Key Skills Addressed	• Using Node.js built-in EventEmitter to decouple side effects from the main request flow • Designing non-blocking background processing without external message queue infrastructure • Understanding the Node.js event loop at a practical, applied level • Comparing synchronous and asynchronous execution using timestamp evidence
Learning Outcome	Students will be able to implement non-blocking, event-driven background processing using Node's built-in EventEmitter and explain how it keeps the main request-response cycle fast.
Lab Session Step-by-Step	

Steps	1. Create a dedicated events.js module:
	`const EventEmitter = require('events');`
	`class TaskEvents extends EventEmitter {}`
	`module.exports = new TaskEvents();`
	2. Register a listener for the task-created event, ideally in a separate listeners.js file:
	`const taskEvents = require('./events');`
	`taskEvents.on('task-created', (task) => {`
	`  console.log(`[Notification] Task "${task.title}" created at ${new Date().toISOString()}`);`
	`});`
	3. Inside the POST /tasks route, emit the event right after saving the task, then respond immediately:
	`const task = await Task.create(req.body);`
	`console.log(`[API] Response sent at ${new Date().toISOString()}`);`
	`res.status(201).json(task);`
	`taskEvents.emit('task-created', task);`
	4. Run the server and create a task via Postman; observe the console log order.
	5. Add an artificial delay inside the listener (e.g., a loop or setTimeout) to make the asynchronous behavior more visible.
	6. Confirm and record that the API response timestamp appears before the listener's completion timestamp in the console.

Tools / Technology / Resources Required	• Node.js (v18+) built-in events module • Express.js, Mongoose • Postman or Thunder Client • Visual Studio Code
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Clarify upfront that this practical replaces a message-queue-based approach (e.g., RabbitMQ/Bull/Redis) which is out of scope for this course EventEmitter is the deliberate, simpler substitute. • Emphasize that emit() does not wait for listeners to finish this is the core concept being tested and is easy to misunderstand as 'asynchronous magic'. • Use the console.log timestamp ordering as the main proof point during evaluation rather than relying on student explanation alone. • Connect this practical back to Practical 9's caching discussion both are examples of decoupling work from the critical request path.

Common Mistakes	• Placing taskEvents.emit() before res.status(201).json(task), which still works but obscures the ordering being demonstrated emit after the response logic is clearer pedagogically. • Writing notification logic directly inside the route handler instead of in a separate listener, defeating the purpose of the exercise. • Not adding any artificial delay or distinguishing log statements, making the synchronous vs asynchronous difference invisible in the console output. • Forgetting to register the listener (taskEvents.on(...)) before the event is ever emitted, causing the event to silently do nothing.

Troubleshooting Guide

Symptom	Likely Cause	Fix
Notification log never appears	Listener file never required/imported, so .on() was never registered	Ensure the listeners file is required once at server startup, e.g., in server.js
API response feels delayed	Listener logic accidentally placed before the response is sent, or emit() called synchronously before res.json()	Move res.status(201).json(task) to execute before taskEvents.emit(...)
Unhandled 'error' event crashes the server	EventEmitter has a special case: an unhandled 'error' event throws	Always register an error listener if emitting custom error events, or avoid using the reserved 'error' event name
Timestamps appear identical	No artificial delay added to the listener to make async timing visible	Add a short setTimeout or loop inside the listener to exaggerate the timing difference for demonstration

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the Node.js Events documentation before next week. • Add a task-deleted event with its own listener and distinct log message. • Add an error event listener that catches and logs any error thrown inside the task-created handler.

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 4/5/9, updated with new commits • Working EventEmitter-based notification system with at least one custom event • Console log evidence (screenshot or recording) showing response-before-handler timestamp ordering • At least one commit specifically for event-driven async processing

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**